

WorkFlow

Сквозной проект по Frontend / Nuxt 4

Полное техническое задание на 9 учебных уровней

Цель проекта

За 9 последовательных уровней превратить учебный репозиторий в полноценное веб-приложение для управления сервисными заявками. Финальный результат должен быть пригоден как демонстрационный проект в портфолио и при этом оставаться учебным: сложность растёт постепенно, требования фиксированы, а каждая новая работа изменяет уже существующую кодовую базу.

Целевой стек: Nuxt 4, Vue 3, TypeScript, Nitro, PostgreSQL, Pinia, Vitest, Playwright, Docker.

Формат: индивидуальная работа. Все решения, не заданные явно в ТЗ, студент выбирает самостоятельно и обосновывает.

Содержание

- 1. Контекст и цель продукта
- 2. Пользователи и роли
- 3. Предметная область и бизнес-правила
- 4. Технологические ограничения
- 5. Общая карта продукта
- 6. Общие требования к качеству
- 7. Девять уровней реализации
- 8. Правила Anti-AI и защита
- 9. Формат сдачи и репозитория
- 10. Финальный релиз для портфолио

1. Контекст и цель продукта

Компания оказывает сервисные услуги корпоративным клиентам. Сейчас заявки поступают через email, телефон и мессенджеры. Сотрудникам сложно отслеживать сроки, ответственных, историю общения и результат выполнения работ.

Необходимо разработать WorkFlow — веб-приложение, в котором клиент создаёт заявку, оператор принимает её и назначает исполнителя, исполнитель ведёт работу, а оператор закрывает заявку после подтверждения результата.

Главный учебный принцип

Ты не делаешь девять разных приложений. Ты один раз создаёшь WorkFlow и затем девять раз расширяешь, исправляешь и рефакторишь ту же систему. В финале именно этот репозиторий становится портфолио-проектом.

Что должно получиться к финалу

- Публичная часть: landing, FAQ, help center, статьи, login/register.
- Авторизованная часть: dashboard, заявки, детали заявки, задачи, клиенты, уведомления, профиль.
- Административная часть: пользователи, категории, SLA, аналитика.
- Собственный серверный API на Nitro и PostgreSQL.
- Аутентификация и авторизация по ролям.
- SSR/SEO для публичных страниц.
- Тесты критических пользовательских сценариев.
- Docker-сценарий локального запуска.
- README с архитектурой, screenshots, ссылкой на demo и описанием решений.

2. Пользователи и роли

Роль	Кто это	Что может делать
Client	Представитель клиента	Создавать свои заявки, видеть свои заявки, комментировать, прикладывать файлы, подтверждать решение, оценивать сервис.
Operator	Сотрудник поддержки	Видеть все заявки, назначать исполнителя, менять приоритет, управлять SLA, закрывать заявки после подтверждения.
Agent	Исполнитель	Видеть назначенные заявки, менять рабочий статус, писать комментарии, фиксировать результат.
Admin	Администратор системы	Управлять пользователями, категориями, SLA и системными настройками.

Правило авторизации

Нельзя полагаться только на скрытие кнопок в интерфейсе. Сервер обязан самостоятельно проверять права пользователя для каждой защищённой операции.

3. Предметная область и бизнес-правила

3.1. Основные сущности

Сущность	Минимальные поля
User	id, name, email, role, status, createdAt

Сущность	Минимальные поля
Customer	id, name, email, phone, company, createdAt
Request	id, title, description, status, priority, customerId, assigneeId, categoryId, slaDueAt, createdAt, updatedAt, closedAt
Comment	id, requestId, authorId, body, createdAt
Category	id, name, description, isActive
Attachment	id, requestId, filename, mimeType, size, url, uploadedBy, createdAt
Notification	id, userId, type, title, body, readAt, createdAt
SLA Policy	id, priority, responseMinutes, resolutionMinutes, isActive

3.2. Статусы заявки

new → assigned → in_progress → waiting → resolved → closed
 ↘ escalated ↗

3.3. Допустимые переходы

Из	В	Условие
new	assigned	Назначен исполнитель.
assigned	in_progress	Исполнитель начал работу.
in_progress	waiting	Обязательно указать причину ожидания.
waiting	in_progress	Можно продолжить работу.
in_progress	resolved	Исполнитель заполнил результат решения.
resolved	closed	Клиент подтвердил решение или оператор закрыл заявку вручную с указанием причины.
new / assigned / in_progress	escalated	Только Operator/Admin. Обязательна причина эскалации.
escalated	assigned	После назначения нового исполнителя.

3.4. Бизнес-правила

- Клиент видит только заявки, связанные с его customer-профилем.
- Agent видит заявки, назначенные ему. Operator и Admin могут видеть все заявки.
- Нельзя закрыть заявку из new напрямую.
- Для перехода в waiting обязательна причина.
- Для перехода в resolved обязательна непустой текст результата решения.
- После closed редактирование описания заявки запрещено.
- Удаление заявок пользователями запрещено. Вместо удаления используется архивирование/скрытие в административном интерфейсе.
- Приоритет Critical/High/Normal/Low влияет на SLA.
- Нельзя назначить деактивированного пользователя исполнителем.
- Комментарий должен иметь автора и не может быть пустым после trim.

3.5. Приоритеты и SLA

Приоритет	Время реакции	Время решения
Critical	15 минут	1 час
High	1 час	4 часа
Normal	4 часа	24 часа
Low	8 часов	72 часа

На уровне 8 значения SLA должны стать редактируемыми через административный раздел. Базовые значения на уровнях 4–7 можно хранить в конфигурации или seed-данных.

4. Технологические ограничения

Проект выполняется на современном стеке Nuxt 4. В актуальной документации Nuxt 4 проект по умолчанию разделяет app-, server-, shared- и configuration-контексты; server/ используется для server handlers/API, shared/ — для кода, который безопасно используется и Vue-приложением, и Nitro. Layers предназначены для переиспользуемого кода и организации крупных приложений. (см. официальную документацию Nuxt 4)

Обязательный стек

Область	Технология / правило
Framework	Nuxt 4
UI	Vue 3 + Composition API + <script setup>
Language	TypeScript
Server	Nuxt server/Nitro
Database	PostgreSQL
State	Pinia — только там, где действительно нужен app-level state
Validation	Zod или эквивалентная типобезопасная схема валидации
Unit/Integration	Vitest
E2E	Playwright
CSS	CSS/Tailwind CSS; UI-kit запрещён как основной источник интерфейса
Quality	ESLint + typecheck
Environment	Docker Compose для финального проекта
Git	Feature branches или осмысленная линейная история с понятными commit messages

Что запрещено

- Копировать готовый проект или tutorial целиком.
- Использовать генератор админ-панелей, который скрывает основную бизнес-логику.
- Подключать готовый UI-kit вместо самостоятельной работы с компонентами там, где это проверяется заданием.
- Хранить секреты, пароли и токены в репозитории.
- Обходить авторизацию только скрыванием элементов интерфейса.
- Оставлять в финальном проекте TODO/placeholder вместо работающей функциональности.

5. Общая карта продукта

5.1. Маршруты

```

/
/login
/register
/faq
/help
/articles
/articles/:slug

/dashboard
/requests
/requests/:id
/tasks
/customers
/customers/:id
/notifications
/profile

/admin/users
/admin/categories
/admin/sla
/admin/analytics

```

5.2. Серверный API к финальному уровню

```

POST /api/auth/login
POST /api/auth/logout
GET /api/auth/me

GET /api/requests
POST /api/requests
GET /api/requests/:id
PATCH /api/requests/:id
POST /api/requests/:id/comments
POST /api/requests/:id/attachments
POST /api/requests/:id/transition

GET /api/customers
GET /api/users
GET /api/categories
GET /api/notifications
PATCH /api/notifications/:id/read

GET /api/admin/users
PATCH /api/admin/users/:id
GET /api/admin/sla
PUT /api/admin/sla/:priority
GET /api/admin/analytics

```

Допускается изменить API по мере развития архитектуры. Изменения должны быть зафиксированы в документации и не нарушать бизнес-контракт приложения.

6. Общие требования к качеству

Область	Минимальное требование к финалу
Type safety	TypeScript strict; отсутствие намеренных any; typecheck проходит.
UX states	У ключевых async-операций есть loading, success, empty и error состояния.
Accessibility	Формы имеют labels; интерактивные элементы доступны с клавиатуры; видимый focus; ошибки доступны пользователю.
Responsive	Основные страницы корректно работают от 360px до desktop.
Errors	Сетевые и бизнес-ошибки не приводят к белому экрану.
Security	Права проверяются на сервере; секреты не попадают в клиентский bundle.
Testing	Критические бизнес-правила покрыты unit/integration тестами, главные пользовательские сценарии — E2E.
Performance	Есть измерение и зафиксированные performance budgets.
Documentation	README, архитектурная схема, setup и описание решений присутствуют.

Уровень 1. UI и компонентное мышление

Цель уровня

Создать первую работающую версию WorkFlow без реального backend. Данные пока локальные.

Задачи

1. Создать Nuxt 4 проект и настроить базовую структуру app/.
2. Сделать default layout приложения и отдельный публичный layout.
3. Реализовать страницы /, /login, /dashboard, /requests, /requests/:id, /profile.
4. Создать список заявок с локальными данными минимум из 12 заявок.
5. Создать страницу деталей заявки с timeline и комментариями.
6. Реализовать фильтры по статусу и приоритету на локальном состоянии.
7. Добавить mobile/tablet/desktop адаптацию.
8. Для списков и карточек сделать отдельные переиспользуемые компоненты.
9. Добавить состояния empty, loading и error хотя бы для одного mock-flow, чтобы показать модель UI states.
10. Добавить базовую keyboard navigation для меню, фильтров и модального окна.

Критерии приёмки

- Все маршруты открываются без console errors.
- Список заявок отображает не менее 12 записей.
- Фильтры реально меняют видимый список.
- Карточка заявки не содержит копипасты между страницами.
- Layout не ломается на 360px, 768px и 1280px.
- Все основные кнопки и ссылки имеют понятное состояние hover/focus/disabled.
- Есть 404/неизвестный маршрут и понятный UI ошибки.

Anti-AI и формат проверки

- Студент получает изображения экранов, а не готовую Figma-структуру.
- Преподаватель на проверке меняет ширину окна и подставляет длинные заголовки/длинные имена.
- На защите студент должен объяснить минимум 5 компонентных решений.

Оценивание — 100 баллов

Критерий	Баллы
Верстка и responsive	20
Vue-компоненты	20
Nuxt/routing/layouts	10
UI states	10
Accessibility	10
Code structure	10
UX/detail	10
Защита	10
Итого	100

Минимальный проходной порог

70/100. Обязательное условие: студент способен устно объяснить собственный код и ключевые инженерные решения. Нельзя компенсировать отсутствие понимания количеством автоматически сгенерированного кода.

Уровень 2. Реальные данные и API

Цель уровня

Заменить локальные mock-данные на API и научиться корректно переживать медленную/ошибочную сеть.

Задачи

1. Подключить GET /api/requests, /api/users, /api/customers и /api/categories из учебного API.
2. Перевести список и карточку заявки на useFetch/useAsyncData.
3. Добавить query-параметры поиска, фильтрации, сортировки и pagination.
4. Синхронизировать состояние фильтров с URL.
5. Добавить debounce для поиска.
6. Сделать skeleton loading, empty state и error state.
7. Добавить retry для временной ошибки загрузки.
8. Описать API-модели TypeScript-типами; не использовать any для ответа API.
9. Сделать нормализацию/адаптацию данных в одном месте, а не в каждом компоненте.

Критерии приёмки

- Прямая ссылка /requests?status=open&page=2 восстанавливает состояние фильтров.
- При задержке API пользователь видит skeleton, а не пустую страницу.
- При 429 или 500 пользователь получает понятную ошибку и может повторить запрос.
- При пустом результате есть отдельный empty state.
- Поиск не отправляет запрос на каждый символ без debounce.
- Отсутствие картинки/названия в одной записи не ломает весь список.

Anti-AI и формат проверки

- API на проверке подменяется на mock с задержкой 3 секунды и последовательностью 429/500/empty.
- Преподаватель меняет формат одного поля ответа; студент должен локализовать место адаптации данных.
- На защите студент объясняет, чем SSR data fetching отличается от обычного client-only fetch в данном сценарии.

Оценивание — 100 баллов

Критерий	Баллы
Data fetching	20
TypeScript	15
HTTP/error handling	15
Query/filter state	15
Composables	10
UX states	10
Robustness	5
Защита	10
Итого	100

Минимальный проходной порог

70/100. Обязательное условие: студент способен устно объяснить собственный код и ключевые инженерные решения. Нельзя компенсировать отсутствие понимания количеством автоматически сгенерированного кода.

Уровень 3. Состояние приложения и взаимодействия

Цель уровня

Добавить реальные изменения заявок и научиться разделять server state, URL state, local state и app state.

Задачи

1. Добавить Pinia там, где состояние должно использоваться несколькими частями приложения.
2. Реализовать изменение статуса, приоритета и исполнителя.
3. Реализовать добавление комментария.
4. Добавить optimistic update для изменения приоритета.
5. При ошибке API выполнять rollback к предыдущему состоянию.
6. Сделать Undo для последнего изменения статуса/приоритета.
7. Сохранять пользовательские настройки интерфейса локально.
8. Определить, какие фильтры относятся к URL state, а какие к UI state.
9. Не допускать одного глобального store для всей предметной области.

Критерии приёмки

- Изменение приоритета визуально происходит без ожидания сети.
- При ответе 409 или 500 приложение откатывает optimistic change и показывает причину.
- Undo восстанавливает предыдущее значение минимум для последнего изменения.
- Обновление страницы не ломает URL-фильтры и пользовательские UI-настройки.
- Неизменяемые данные не дублируются в нескольких источниках без объяснения.

Anti-AI и формат проверки

- Преподаватель делает PATCH медленным и возвращает 409 Conflict.
- Вопрос на защите: почему конкретное состояние лежит в Pinia, URL или локальном компоненте?
- После сдачи добавляется представление "Мои заявки" без права переписать store с нуля.

Оценивание — 100 баллов

Критерий	Баллы
State model	20
Pinia	15
Optimistic UI/rollback	15
CRUD flows	15
URL/local persistence	10
Composables	10
Maintainability	5
Защита	10
Итого	100

Минимальный проходной порог

70/100. Обязательное условие: студент способен устно объяснить собственный код и ключевые инженерные решения. Нельзя компенсировать отсутствие понимания количеством автоматически сгенерированного кода.

Уровень 4. Full-stack Nuxt: Nitro, PostgreSQL, auth

Цель уровня

Перенести backend в Nitro и сделать WorkFlow полноценным full-stack приложением.

Задачи

1. Поднять PostgreSQL и описать схему данных.
2. Реализовать server/api для auth, requests, users, customers, categories и comments.
3. Подключить seed-данные для разработки.
4. Сделать login/logout/session.
5. Ввести роли Client, Operator, Agent, Admin.
6. Проверять права на сервере каждой защищённой операции.
7. Реализовать создание и изменение заявок через server API.
8. Реализовать бизнес-правила переходов статусов.
9. Вынести совместно используемые чистые типы и utilities в shared/ там, где это оправдано.
10. Не импортировать app-only код в server и server-only код в app.

Критерии приёмки

- После перезапуска приложения данные сохраняются в PostgreSQL.
- Незалогиненный пользователь не получает приватные данные.
- Client не может запросить чужую заявку через прямой API-вызов.
- Agent не может менять системные настройки.
- Invalid status transition возвращает понятный API error.
- Нельзя назначить деактивированного пользователя.
- Ошибки API используют осмысленные HTTP status codes.

Anti-AI и формат проверки

- Преподаватель тестирует authorization напрямую через API, минуя UI.
- Два параллельных запроса пытаются назначить разные значения одного поля.
- Студент должен показать, где находится server-only код и почему он недоступен клиенту.

Оценивание — 100 баллов

Критерий	Баллы
Nitro/API	15
PostgreSQL/data model	15
Auth	15
Authorization	15
Business rules	20
Validation	10
Error handling	5
Boundary understanding	5
Итого	100

Минимальный проходной порог

70/100. Обязательное условие: студент способен устно объяснить собственный код и ключевые инженерные решения. Нельзя компенсировать отсутствие понимания количеством автоматически сгенерированного кода.

Уровень 5. Debugging, testing и надёжность

Цель уровня

Научиться находить и исправлять проблемы в существующем проекте, а не только добавлять новые функции.

Задачи

1. Получить репозиторий текущей версии WorkFlow с набором намеренных багов.
2. Для каждого бага описать шаги воспроизведения и предполагаемую причину.
3. Исправить race condition двойной отправки комментария.
4. Исправить проблему устаревшего search response.
5. Исправить приватный route после logout.
6. Исправить потерю/рассинхронизацию состояния после refresh.
7. Исправить некорректное поведение при API 500/401/409.
8. Добавить unit/integration tests для business rules и composables.
9. Добавить Playwright E2E для login → create request → assign → change status → comment → logout.
10. Добавить regression tests на каждый исправленный баг.

Критерии приёмки

- Каждый заявленный баг воспроизводится до исправления и закрыт тестом после исправления.
- Ключевые E2E-сценарии проходят в чистом окружении.
- Тесты не привязаны только к реализации; они проверяют ожидаемое поведение.
- После исправлений typecheck и lint проходят.
- В проекте нет knowingly flaky тестов.

Anti-AI и формат проверки

- Студент получает только описание симптома, а не root cause.
- Преподаватель запускает дополнительный сценарий, не указанный в списке багов.
- На защите нужно объяснить, почему выбранный тест действительно ловит регрессию.

Оценивание — 100 баллов

Критерий	Баллы
Debugging/root cause	25
Unit/integration tests	15
E2E	20
Regression coverage	15
Async/race handling	10
Code quality	10
Защита	5
Итого	100

Минимальный проходной порог

70/100. Обязательное условие: студент способен устно объяснить собственный код и ключевые инженерные решения. Нельзя компенсировать отсутствие понимания количеством автоматически сгенерированного кода.

Уровень 6. SSR, SEO, hydration и performance

Цель уровня

Сделать публичную часть продукта быстрой, индексируемой и предсказуемой при SSR.

Задачи

1. Добавить landing, FAQ, help center и статьи.
2. Сделать публичные страницы SSR-first.
3. Добавить title, description, canonical и Open Graph.
4. Добавить sitemap и robots.txt.
5. Добавить корректные 404 для статей и сущностей.
6. Провести performance baseline на заданном наборе страниц.
7. Исправить переданный hydration mismatch.
8. Оптимизировать изображения, lazy loading и initial JS.
9. Использовать подходящий режим рендеринга для разных страниц; при необходимости применить routeRules.
10. Зафиксировать результат в PERFORMANCE.md.

Критерии приёмки

- Публичные страницы отдают содержательный HTML при первом запросе.
- Metadata уникальны и соответствуют странице.
- Нет hydration mismatch в консоли на целевых страницах.
- Performance report содержит before/after и объясняет причину каждого изменения.
- Заданный performance budget соблюден или есть обоснованный документированный компромисс.

Anti-AI и формат проверки

- Преподаватель предоставляет намеренно плохой baseline и не сообщает bottleneck.
- Нужно добиться улучшения минимум по двум независимым причинам, а не только "включить lazy".
- Одну оптимизацию преподаватель временно откатывает и просит объяснить последствия.

Оценивание — 100 баллов

Критерий	Баллы
SSR/rendering model	15
SEO	15
Hydration	15
Performance diagnosis	20
Optimization	20
Caching/route rules	10
Защита	5
Итого	100

Минимальный проходной порог

70/100. Обязательное условие: студент способен устно объяснить собственный код и ключевые инженерные решения. Нельзя компенсировать отсутствие понимания количеством автоматически сгенерированного кода.

Уровень 7. Архитектура и рефакторинг

Цель уровня

Привести растущий WorkFlow к архитектуре, которую можно дальше поддерживать без постоянного увеличения связности.

Задачи

1. Провести аудит текущей структуры и выделить архитектурные проблемы.
2. Разделить домены requests, customers, users, comments, auth, notifications.
3. Определить правила зависимостей между слоями.
4. Отделить UI, orchestration/composables, data access и business rules там, где это нужно.
5. Выделить shared/types и shared/utils только для действительно общего и безопасного кода.
6. При необходимости использовать Nuxt Layers для повторно используемой или административной части.
7. Убрать дублирование и скрытые циклические зависимости.
8. Составить ARCHITECTURE.md с диаграммой и ADR-lite для трёх решений.

Критерии приёмки

- Новая структура объяснима по ответственности, а не только по типу файла.
- Бизнес-правило перехода статуса не размазано по нескольким UI-компонентам.
- Изменение API не требует редактировать десятки визуальных компонентов.
- Общие utilities не зависят от Vue/Nitro, если они помещены в shared/.
- Критическая функциональность после рефакторинга покрыта существующими тестами.

Anti-AI и формат проверки

- ARCHITECTURE.md сдается до рефакторинга.
- Преподаватель выбирает случайный dependency path и просит объяснить его.
- Одна архитектурная проблема намеренно оставляется спорной: студент должен выбрать решение и защитить trade-off.

Оценивание — 100 баллов

Критерий	Баллы
Architecture	25
Separation of concerns	15
Refactoring safety	20
Type system	10
Maintainability	15
Documentation	5
Защита	10
Итого	100

Минимальный проходной порог

70/100. Обязательное условие: студент способен устно объяснить собственный код и ключевые инженерные решения. Нельзя компенсировать отсутствие понимания количеством автоматически сгенерированного кода.

Уровень 8. Эволюция продукта и изменение требований

Цель уровня

Научиться развивать существующую систему под реальные изменения бизнеса без переписывания проекта.

Задачи

1. Добавить настраиваемый SLA для каждого приоритета через admin.
2. Добавить статус escalated с обязательной причиной.
3. Добавить вложения с ограничением размера, количества и типов файлов.
4. Добавить уведомления о назначении, эскалации и приближении SLA.
5. Добавить административную аналитику по статусам, SLA и загрузке исполнителей.
6. Обновить database schema и seed/fixtures.
7. Обновить API, UI и tests во всех затронутых местах.
8. Сохранить работоспособность уже реализованных пользовательских сценариев.

Критерии приёмки

- Каждое change request реализовано через изменения существующих доменов, а не через дублирование функциональности.
- Старые E2E-сценарии продолжают проходить.
- Новые бизнес-правила покрыты тестами.
- Изменения схемы данных воспроизводимы из чистого окружения.
- Git history показывает последовательную эволюцию проекта.

Anti-AI и формат проверки

- Преподаватель выдаёт change requests по одному после сдачи предыдущей итерации.
- Переписывание проекта после каждого требования запрещено и отслеживается по Git history.
- Один change request выдаётся в изменённой формулировке прямо на защите.

Оценивание — 100 баллов

Критерий	Баллы
Feature implementation	20
No regressions	20
Architecture under change	20
Tests	15
Data/API migrations	10
Git history	5
Защита	10
Итого	100

Минимальный проходной порог

70/100. Обязательное условие: студент способен устно объяснить собственный код и ключевые инженерные решения. Нельзя компенсировать отсутствие понимания количеством автоматически сгенерированного кода.

Уровень 9. Production release и портфолио

Цель уровня

Довести WorkFlow до состояния самостоятельного Middle-проекта: он должен стабильно запускаться, быть документированным, протестированным и объяснимым автором.

Задачи

1. Провести полный release audit: функциональность, UX, security basics, accessibility, performance и code quality.
2. Настроить Docker Compose для приложения и PostgreSQL.
3. Сделать reproducible setup для нового разработчика.
4. Проверить env/config и удалить секреты из репозитория.
5. Подготовить seed/demo account без использования реальных персональных данных.
6. Добавить финальный набор E2E critical paths.
7. Проверить keyboard navigation и формы.
8. Провести финальный performance audit и зафиксировать метрики.
9. Привести README к портфолио-формату: проблема, решение, стек, архитектура, demo, screenshots, testing, performance, что было самым сложным.
10. Подготовить 5-минутное demo video или live demo сценарий.
11. Подготовить ARCHITECTURE.md и TRADEOFFS.md.
12. Создать финальный release tag и clean main branch.

Критерии приёмки

- Новый разработчик может клонировать репозиторий и запустить проект по README.
- Проект стартует в чистом окружении через один документированный сценарий.
- Критические E2E проходят.
- Публичные страницы и dashboard не имеют blocker-level ошибок.
- Нет секретов в репозитории.
- README содержит ссылку на demo и понятную архитектурную схему.
- Проект можно показать работодателю без устных пояснений о том, как его запускать.

Anti-AI и формат проверки

- Во время защиты преподаватель добавляет один скрытый баг в тестовую ветку и просит найти root cause.
- На защите студент получает один change request и 10 минут на проектирование решения без написания кода.
- Преподаватель открывает случайный файл и просит пройти путь данных до API.
- Финальная оценка зависит не только от работающего UI, но и от способности объяснить собственные решения.

Оценивание — 100 баллов

Критерий	Баллы
Functional completeness	20
Vue/Nuxt	10
TypeScript	10
Architecture	15
Full-stack	10
State management	10
Testing	10

Критерий	Баллы
Performance	5
Accessibility	5
Защита	5
Итого	100

Минимальный проходной порог

70/100. Обязательное условие: студент способен устно объяснить собственный код и ключевые инженерные решения.
Нельзя компенсировать отсутствие понимания количеством автоматически сгенерированного кода.

8. Правила Anti-AI и защиты

Использование AI не запрещается. Запрещается выдавать сгенерированный результат за собственное понимание. AI можно использовать как инструмент: для поиска документации, проверки идеи, объяснения ошибки или генерации рутинного boilerplate. Ответственность за архитектуру, корректность, безопасность и объяснимость результата несёт студент.

8.1. Что студент фиксирует

- AI_USED.md: для каких задач использовались AI-инструменты и что именно было принято/отклонено.
- В commit history не должно быть скрытия этапов разработки через один финальный commit.
- Архитектурные решения должны иметь человеческое обоснование в ARCHITECTURE.md/TRADEOFFS.md.

8.2. Что преподаватель может сделать

- Подменить API поведением, которое раньше не демонстрировалось.
- Задать новый edge case.
- Попросить быстро найти причину бага в знакомом коде.
- Попросить объяснить случайный файл, composable, endpoint или путь данных.
- Дать небольшое change request и попросить спроектировать изменение без написания кода.

Принцип оценки

Работающий код без понимания не равен владению компетенцией. В спорных случаях оценка снижается за отсутствие объяснения, даже если UI/тесты формально проходят.

9. Формат сдачи и репозитория

9.1. Рекомендуемая структура

```

app/
  components/
  composables/
  layouts/
  middleware/
  pages/
  plugins/
  utils/
server/
  api/
  middleware/
  utils/
shared/
  types/
  utils/
public/
content/
  articles/

tests/
  unit/
  e2e/
docs/
  ARCHITECTURE.md
  TRADEOFFS.md
  PERFORMANCE.md
  README.md
  AI_USED.md
  Dockerfile
  compose.yaml
  nuxt.config.ts
  package.json

```

Конкретная структура может отличаться. Она должна быть объяснена и не должна нарушать контекстные границы Nuxt 4.

9.2. Каждый stage commit

- В начале этапа создать ветку или зафиксировать понятную точку старта.
- В ходе работы делать небольшие осмысленные commits.
- После завершения этапа создать tag или release-point: level-1, level-2, ... level-9.
- Не коммитить .env с секретами.

9.3. Артефакты каждого уровня

Уровень	Что сдаётся
1	Git repository + короткий README + screenshots.
2	Repository + API notes + описание error/loading states.
3	Repository + state map/notes.
4	Repository + DB schema/ERD + API list + seed instructions.
5	Repository + bug reports + tests + regression list.
6	Repository + PERFORMANCE.md + before/after measurements.
7	Repository + ARCHITECTURE.md + dependency diagram + ADR-lite.
8	Repository + change log + migration notes + updated tests.
9	Portfolio-ready repository + README + demo + architecture + tests + performance + AI_USED.md.

10. Финальный релиз для портфолио

10.1. README должен отвечать на вопросы

- Какую проблему решает WorkFlow?
- Кто пользователи и какие у них роли?
- Какие ключевые сценарии поддерживаются?
- Почему выбран Nuxt 4?
- Как устроены app/server/shared контексты?
- Где находится бизнес-логика?
- Как устроен state management?
- Как тестируется критический workflow?
- Какие performance решения были приняты?
- Какие компромиссы и ограничения остались?

10.2. Финальный demo сценарий

1. Войти под Client и создать заявку.
2. Показать заявку в списке и на странице деталей.
3. Войти под Operator и назначить исполнителя.
4. Войти под Agent и перевести заявку в работу.
5. Добавить комментарий и результат решения.
6. Вернуть роль Operator и закрыть заявку.
7. Показать уведомление и историю заявки.
8. Открыть Admin и показать SLA/analytics.
9. Показать одну unit-тестируемую бизнес-функцию и один E2E-сценарий.
10. Показать performance report и объяснить один trade-off.

10.3. Что считается готовым портфолио-проектом

Признак	Готово, если
Product	Понятно, какую реальную задачу решает приложение.
Demo	Есть рабочая ссылка или воспроизводимый локальный запуск.
UI	Интерфейс выглядит как единый продукт, а не набор экранов.
Engineering	Есть backend, DB, auth, validation, tests и error handling.
Architecture	Структура проекта объяснима и поддерживаема.
Performance	Есть измерения, а не только субъективное "быстро".

Признак	Готово, если
Documentation	Новый разработчик понимает, как запустить и продолжить проект.
Defense	Автор способен пройти путь данных и объяснить ключевые решения.

Граница курса

Финальный результат программы — уверенный Middle Frontend Developer в рамках заявленного стека. Курс проверяет самостоятельную инженерную работу с достаточно сложным существующим приложением, а не управление командой или обязанности Tech Lead.

Приложение А. Минимальные вопросы для защиты

- Где в проекте проходит граница app/server/shared и почему?
- Почему выбран именно такой способ хранения фильтров?
- Как проходит запрос от клика пользователя до БД и обратно?
- Что произойдёт при 401/403/409/500?
- Где реализована проверка права пользователя на чтение заявки?
- Как предотвращается двойная отправка операции?
- Как работает rollback optimistic update?
- Почему один из модулей выделен в composable/store/service?
- Как вы воспроизвели самый сложный баг?
- Какая оптимизация дала наибольший выигрыш и как вы это измерили?
- Как изменится архитектура при появлении второго клиента — например, мобильного?
- Какое архитектурное решение вы бы сейчас пересмотрели и почему?

Приложение В. Контрольный чек-лист студента

- Репозиторий доступен преподавателю.
- Проект устанавливается по README.
- В репозитории нет секретов.
- Есть seed/demo данные.
- Есть typecheck.
- Есть lint.
- Есть unit/integration tests.
- Есть E2E critical paths.
- Основные страницы работают на mobile.
- Основные формы доступны с клавиатуры.
- Ошибки API отображаются пользователю.
- README содержит screenshots и demo.
- ARCHITECTURE.md актуален.
- PERFORMANCE.md актуален.
- AI_USED.md заполнен честно.
- Git history показывает развитие проекта.
- Финальный demo сценарий воспроизводится с чистого окружения.

Примечание о технологической базе

Технологическая база соответствует актуальной документации Nuxt 4: `app/` для приложения, `server/` для server handlers/API, `shared/` для безопасно разделяемого кода и `layers/` для переиспользования. Перед стартом потока преподавателю рекомендуется зафиксировать версии Node, Nuxt и зависимостей в lockfile starter-репозитория.

Приложение С. Регламент дистанционной приемки

Назначение. Этот раздел является обязательной частью ТЗ для дистанционного обучения. Он задаёт единый способ, которым преподаватель может принять работу студента удалённо, не клонируя проект локально и не полагаясь только на демонстрационное видео.

Главный принцип

Каждый уровень должен быть проверяем по четырём независимым источникам: (1) работающий demo URL, (2) Git-репозиторий и история изменений, (3) автоматические отчёты CI/тестов, (4) короткая устная защита. Ни один из четырёх источников не заменяет остальные.

С.1. Что студент обязан предоставить преподавателю

Артефакт	Обязательное содержимое	Как проверяет преподаватель
Demo URL	Рабочий HTTPS URL; публичная часть доступна без авторизации; авторизованная часть — через тестовые аккаунты.	Открывает в браузере без установки проекта.
Repository URL	GitHub/GitLab/Bitbucket; доступ преподавателю; ветка main/master соответствует последней принятой версии.	Смотрит код, commit history, tags и README.
Demo accounts	Отдельные аккаунты для client/operator/agent/admin; пароли не являются реальными персональными данными.	Входит по очереди в браузер и проверяет роли.
README	Установка, переменные окружения, demo URL, роли, архитектура, сценарий демонстрации, известные ограничения.	Проверяет, можно ли понять проект без устных пояснений.
CI report	Typecheck, lint, unit/integration, E2E; ссылки на успешные job/run.	Открывает последний успешный pipeline.
Документация уровня	ARCHITECTURE.md, TRADEOFFS.md, PERFORMANCE.md, AI_USED.md — в зависимости от уровня.	Проверяет соответствие заявленным решениям.
Demo video	5–8 минут; экран + голос; без монтажа, скрывающего работу приложения.	Используется как резерв и как доказательство общего сценария.
Release tag	level-N или финальный v1.0.0.	Позволяет однозначно определить принятую версию.

С.2. Требование «преподаватель не должен настраивать окружение»

С уровня 4 и выше основной способ проверки — удалённый demo URL. Локальный запуск проекта преподавателем допускается только как дополнительная проверка. Если преподаватель должен устанавливать Node, PostgreSQL, Docker, миграции и другие зависимости вручную только для того, чтобы убедиться, что приложение работает, это считается недостатком сдачи.

- Demo URL должен работать по HTTPS.
- Для авторизации используются только учебные аккаунты.
- В системе должны быть seed/demo данные, достаточные для прохождения контрольных сценариев.
- Студент обязан описать способ полного сброса демо-данных. Предпочтительно: отдельная команда/route/seed, не требующая ручного редактирования БД преподавателем.

- Если внешний сервис временно недоступен, студент обязан предоставить резервный способ воспроизведения: локальный mock mode, recorded fixture или видео прохождения сценария.

С.3. Что преподаватель принимает дистанционно на каждом уровне

Уровень	Основная удалённая проверка	Минимальный результат
1	Открыть demo, проверить responsive и 3 случайных состояния компонентов.	Интерфейс работает на 360/768/1280px; компоненты переиспользуются.
2	Открыть каталог, изменить query-параметры, обновить страницу, проверить loading/error/empty.	URL восстанавливает состояние; API-ошибки не ломают приложение.
3	Выполнить изменение заявки, искусственно замедлить запрос или использовать подготовленный mock, проверить optimistic update/rollback.	Состояние приложения синхронно с сервером и не теряется после F5.
4	Войти под 3–4 ролями и попытаться выполнить запрещённые действия.	Авторизация и права проверяются на сервере, не только скрыванием UI.
5	Запустить CI и открыть отчёт Playwright/Vitest; воспроизвести один bug scenario.	Есть автоматические тесты критических путей и regression test на исправленный баг.
6	Открыть публичную страницу, посмотреть source/metadata и Lighthouse/PageSpeed-отчёт; проверить hydration-sensitive сценарий.	SSR/SEO работают, performance budget документирован и достигнут.
7	Открыть random file и пройти dependency/data path; сопоставить с ARCHITECTURE.md.	Студент объясняет, почему код находится именно в этом слое.
8	Попросить реализованный ранее change request и проверить старые E2E.	Новые требования добавлены без разрушения старого поведения.
9	Пройти финальный demo сценарий, выполнить скрытый edge case и короткий change request.	Проект готов к портфолио; студент самостоятельно объясняет и защищает решение.

С.4. Стандартная процедура дистанционной защиты

1. Студент заранее отправляет преподавателю единый Submission URL, из которого доступны repository, demo, CI и документы.
2. Преподаватель открывает demo URL в режиме инкогнито или в отдельном профиле браузера.
3. Студент демонстрирует контрольный пользовательский сценарий без использования devtools, AI-ассистента и заранее подготовленных команд, скрывающих реальные действия.
4. Преподаватель самостоятельно повторяет один контрольный шаг по инструкции из ТЗ.
5. Преподаватель задаёт 3–5 вопросов по случайно выбранным файлам/модулям.
6. Преподаватель даёт один новый edge case или небольшое change request. Студент объясняет решение в течение 5–10 минут; код писать необязательно, если проверяется архитектурное мышление.
7. Преподаватель фиксирует результат по рубрике и сохраняет ссылку/скриншот на принятую версию и CI run.

Запись защиты

Рекомендуется записывать экран и голос 15–25 минут. Видео должно содержать URL проекта, демонстрацию браузера, путь по коду и ответы на вопросы. Запрещено заранее монтировать ответы так, чтобы преподаватель не мог увидеть ход выполнения.

С.5. Протокол скрытой проверки

Чтобы AI не мог полностью заменить понимание проекта, преподаватель выбирает минимум один сценарий из каждой категории:

Категория	Примеры скрытой проверки
Данные	API задерживается; возвращает 404/409/429/500; поле равно null; список пуст.
Состояние	Быстрые повторные клики; два параллельных изменения; F5 во время работы; stale response.
Права	Client открывает чужую заявку; Agent меняет чужую заявку; Operator пытается admin action.
Бизнес-правила	Запрещённый переход статуса; отсутствует обязательная причина; истёк SLA.
UX	Мобильная ширина; keyboard-only; длинный текст; ошибка загрузки файла.
Код	Объяснить случайный composable, endpoint, store или dependency path.
Изменения	Добавить новое правило без полной переписи существующего модуля.

С.6. Как принимать работу, если demo URL временно недоступен

- Один кратковременный инфраструктурный сбой не аннулирует работу. Преподаватель фиксирует время и проверяет второй канал: demo video, CI artefact или локальный fallback.
- Если недоступен именно backend, студент обязан предоставить документированный mock mode либо временную копию demo environment.
- Если проект нельзя проверить ни по одному каналу, оценка по функциональным критериям откладывается до восстановления доступа; при этом код и документы можно оценивать независимо.
- Студент не должен требовать от преподавателя ручного изменения production БД для восстановления demo.

С.7. Анти-AI: что считается самостоятельным выполнением

- AI разрешён как инструмент, но студент обязан понимать каждое принятое решение и уметь воспроизвести критическую функциональность без подсказок AI.
- Критические бизнес-правила должны быть реализованы и защищены студентом; ссылка на генерацию AI не является доказательством корректности.
- В AI_USED.md студент фиксирует: инструмент, задачу, что предложил AI, что студент изменил/отклонил и почему.
- Запрещено скрывать использование AI через удаление commit history, squash всех этапов в один commit или выдавать чужой репозиторий за собственный.
- Преподаватель вправе попросить студента реализовать небольшую изолированную часть на общем экране без AI. Это не должно быть большой feature; достаточно проверить понимание основного паттерна проекта.
- При существенном расхождении между поведением demo и объяснением студента преподаватель оценивает только подтверждённую компетенцию.

С.8. Рубрика дистанционной приемки

Результат	Условия
Зачёт уровня	Не менее 70/100; обязательные критические критерии уровня выполнены;

	защита пройдена.
Условный зачёт	60–69/100; нет критической ошибки в безопасности/данных; преподаватель выдаёт 1 исправление и повторную короткую проверку.
Незачёт	Менее 60/100; критический сценарий не работает; студент не способен объяснить собственный код; нарушена серверная авторизация; скрытая проверка выявляет фундаментальное непонимание.
Финальный проект	Не менее 75/100 на уровне 9 и успешная демонстрация финального сценария + скрытая проверка + защита.

С.9. Формат Submission URL

Студент сдаёт одну страницу или сообщение по шаблону:

Студент: Фамилия Имя

Уровень: 1–9

Release/tag: level-N / v1.0.0

Repository: https://...

Demo: https://...

CI: https://...

Demo accounts: client / operator / agent / admin

README: https://...

ARCHITECTURE: https://...

PERFORMANCE: https://... (с уровня 6)

AI_USED: https://...

Demo video: https://...

Known limitations: ...

С.10. Чек-лист преподавателя перед выставлением оценки

Проверка	Да/нет
Открывается Demo URL	
Работают тестовые аккаунты нужных ролей	
Последний commit/tag соответствует сданной версии	
CI green или студент объяснил конкретный non-blocking failure	
Критический пользовательский сценарий пройден	
Проверены права доступа на сервере	
Проверен минимум один скрытый edge case	
Студент объяснил случайный участок кода	
Проверена Git history	
Документация соответствует реальному проекту	
AI_USED заполнен	
Итоговая оценка зафиксирована	

С.11. Минимальная инфраструктура преподавателя

- Современный браузер с режимом инкогнито/отдельным профилем.
- Доступ к GitHub/GitLab/Bitbucket или учебной системе LMS.
- Видеосвязь и демонстрация экрана для защиты.
- Не требуется локальный Node/PostgreSQL/Docker для основной приемки; они нужны только для выборочного аудита, если преподаватель это использует.
- При большом потоке студентов рекомендуется единая форма Submission URL и таблица с оценками.

Примечание для преподавателя. Дистанционная приемка не должна превращаться в полный code review всего репозитория. Её цель — доказать четыре вещи: продукт работает, критические сценарии тестируются, права и бизнес-правила реализованы на правильном уровне, а студент понимает решения и способен самостоятельно изменить их под новое требование.